

Multiplicação Esparsa Matriz-Vetor para LLMs Podados

Formato Customizado Inspirado no MACKO
com Layout SoA e Precisão Mista FP16/FP64 em GPU CUDA

Prof. Ricardo Augusto Pereira Franco

1895 GB/s

Pico GPU MACKO

14×

Speedup máx. FP16 vs CSR-CUDA

15 configs

$n \in \{4K, 16K, 32K\} \times s \in \{30-90\}\%$

• Motivação

Por que SpMV importa para LLMs modernos?

Inferência é dominada por memória

Razão FLOP/byte < 1 em SpMV — a GPU espera pela memória, não computa.

Pruning cria esparsidade massiva

LLMs podados atingem 30–90% de zeros; CSR padrão não explora esse padrão.

Coalescência é crítica em GPU

Stride de 48 bytes no CSR fragmenta as transações de warp — desperdiça banda.

Memory-Bound

FLOP/byte (SpMV)



Pico GPU (TFLOP/s)



Banda (GB/s)



O gargalo não é computação — é largura de banda de memória.

• Objetivos & Contribuições Originais

O que propomos além de replicar o MACKO

01

Layout SoA

Refatoração de Array of Structs (AoS) para Structure of Arrays (SoA), eliminando o padding silencioso de struct e maximizando coalescência de warp na GPU.

02

Delta Encoding int16

Compressão de índices de coluna de int32 para int16 via endereçamento relativo por chunk — com análise empírica de quando o padding C anula o ganho.

03

Variante FP16/FP64

Valores armazenados em FP16 no device (2 bytes vs 8 bytes), acumulação em FP64. Reduz footprint de vals em 4x, mantendo corretude numérica verificada.

⚡ Meta: superar o CSR-CUDA ingênuo em toda a faixa de esparsidades e bater o cuSPARSE em tempo absoluto nas configurações de alta esparsidade.

• Metodologia

Ambiente de execução e formatos implementados

Hardware & Setup

GPU: NVIDIA RTX 4090 · sm_89 · 24 GB GDDR6X

CUDA: Toolkit 12.0 · nvcc -arch=sm_89 -O3

CPU: gcc -O3 -fopenmp · clock_gettime

Timing GPU: cudaEvent (validado via Nsight Systems, $\Delta < 2\%$)

Profiling: nsys OK · ncu bloqueado (sem sudo)

Amostras: 101 runs · 1 warm-up descartado · 100 amostras

Configs: $n \in \{4096, 16384, 32768\}$ · $s \in \{0.30, 0.50, 0.70, 0.85, 0.90\}$

10 Implementações no Harness

①② CPU sequencial + OpenMP (CSR)

③④ CPU sequencial + OpenMP (SP24)

⑤ GPU CUDA thread/row (CSR)

⑥ GPU cuSPARSE (CSR) — teto industrial

⑦⑧ CPU seq. + OpenMP (MACKO SoA)

⑨ GPU warp/row (MACKO SoA FP64) ★

⑩ GPU warp/row (MACKO SoA FP16) ★

• Evolução do Formato MACKO: v1 → v2 → v3

Três iterações, três lições

v1 — AoS int32

48 bytes/chunk

Baseline

cols[4] em int32 + vals[4] em double. Kernel warp-per-row já entrega 1786 GB/s.



0.017 ms · 1786 GB/s

v2 — AoS int16+mask

48 bytes/chunk (!)

Regressão -18% GPU

Delta encoding int16 + valid_mask. Padding C anula o ganho. Mais instruções, mesma banda.



0.020 ms · 1549 GB/s

v3 — SoA

43 bytes/chunk

Ganho +18% GPU ✓

Arrays separados por campo. Coalescência ideal: 32 lanes, mesma instrução, 32 bytes contíguos.



0.014 ms · 1886 GB/s

Layout SoA + Kernel Warp-per-Row

Por que a combinação é eficaz na GPU

Struct MACKO-SoA

```
uint8_t  *valid_masks;
          [total_chunks] → 1 byte/chunk

int16_t  *col_bases;
          [total_chunks] → 2 bytes/chunk

int16_t  *col_deltas;
          [chunks × 4]   → 8 bytes/chunk

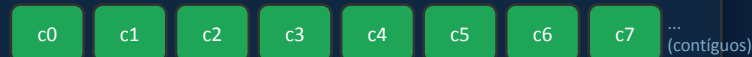
double   *vals;
          [chunks × 4]   → 32 bytes/chunk
```

Total → 43 bytes/chunk ✓
(vs 48 bytes no AoS — padding anulado)

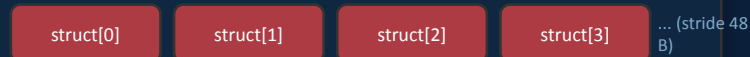
Coalescência de Warp

32 lanes do mesmo warp:

SoA ✓ — 1 transação de 32 bytes



AoS ✗ — múltiplas transações, stride 48 B



Kernel warp-per-row

1 warp (32 lanes) → 1 linha da matriz

Cada lane itera chunks separados → sem divergência SIMT

Redução via `__shfl_down_sync` · tpb=256 (8 warps/bloco)

• Variante de Precisão Mista FP16/FP64

Redução de footprint com corretude preservada

MACKO SoA FP64

valid_masks 1 byte/chunk

col_bases 2 bytes/chunk

col_deltas 8 bytes/chunk

vals (double) 32 bytes/chunk

Total 43 bytes/chunk

MACKO SoA FP16 ★

valid_masks 1 byte/chunk

col_bases 2 bytes/chunk

col_deltas 8 bytes/chunk

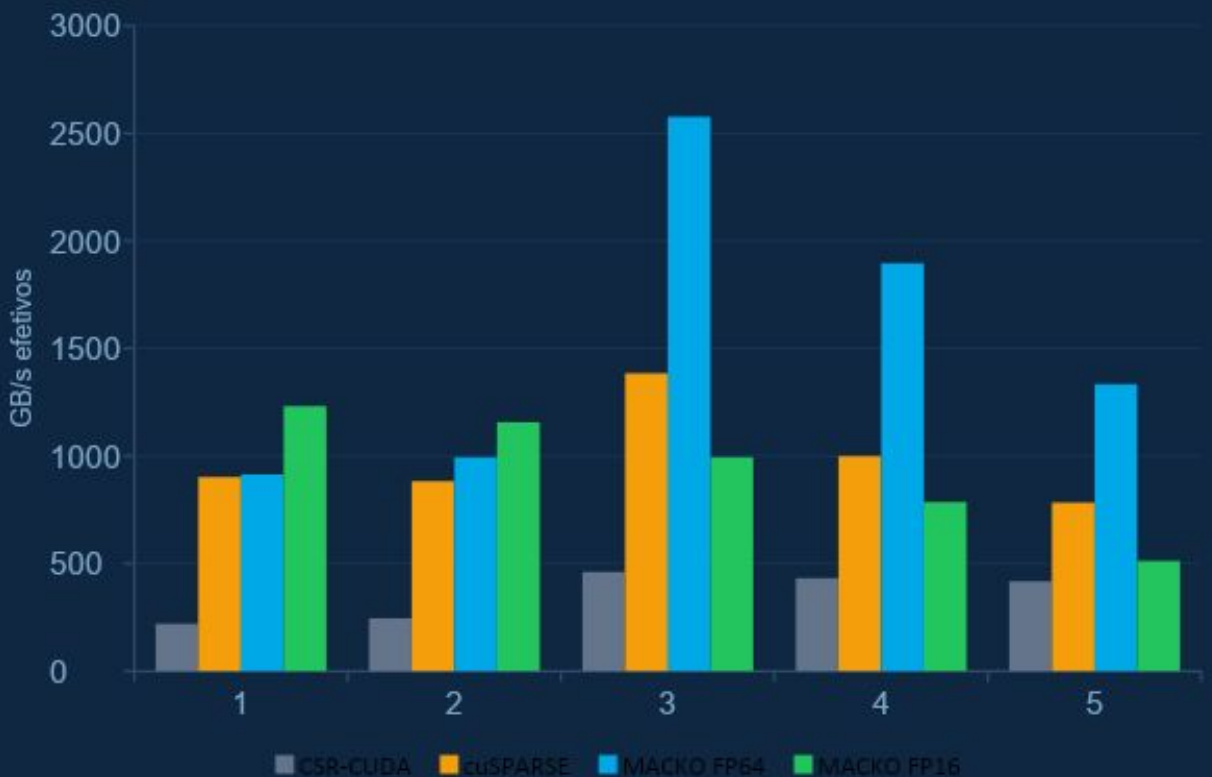
vals (__half) 8 bytes/chunk ↓4×

Total 19 bytes/chunk ↓2.3×

Multiplicação em FP64 após __half2double on-the-fly · Acumulação em double · Corretude verificada com tolerância 10^{-6} em todas as 15 configurações

Resultados — $n=4096$ · GPU (100 amostras cada)

Comparação em 5 esparsidades · RTX 4090



2577 GB/s

MACKO FP64 · $s=0.70$

1895 GB/s

MACKO FP64 · $s=0.85$

1231 GB/s

MACKO FP16 · $s=0.30$

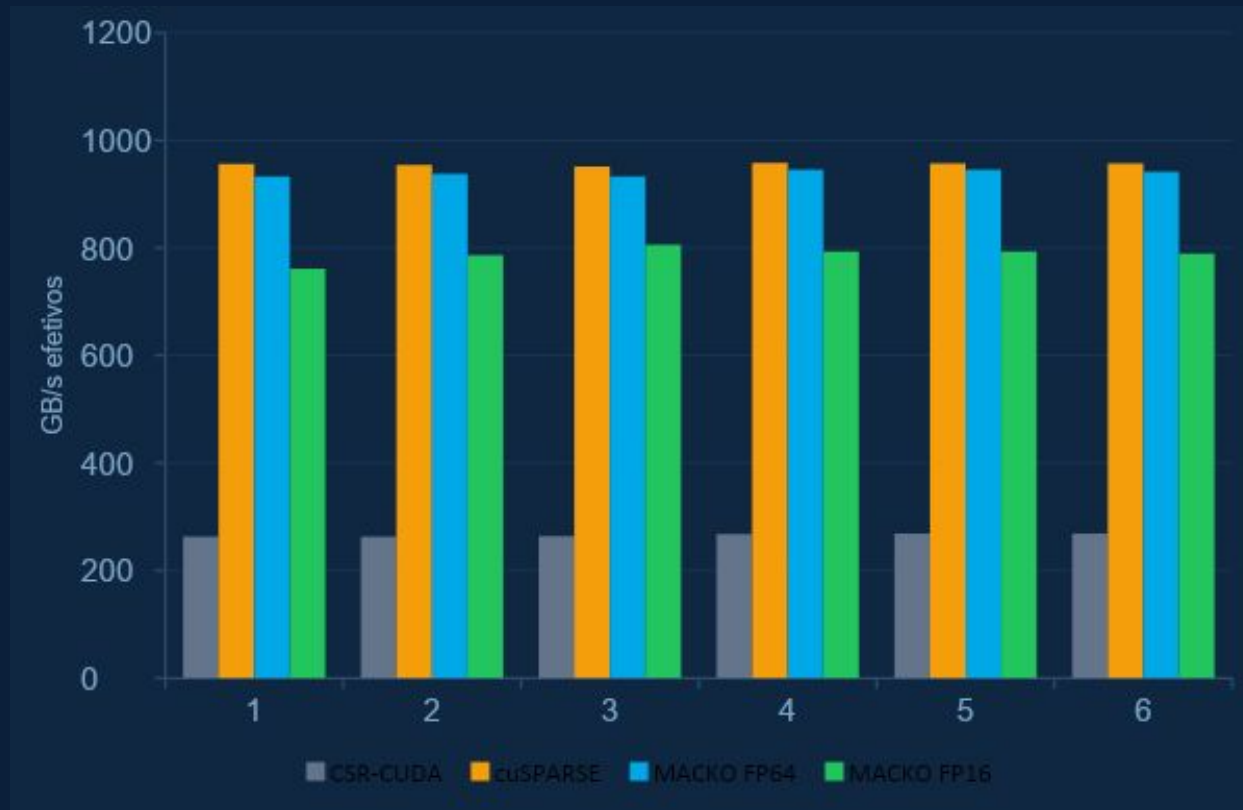
• Speedup vs. Baseline CSR-CUDA

MACKO FP64 e FP16 · múltiplas configurações

Configuração	MACKO FP64 vs CSR-CUDA	MACKO FP16 vs CSR-CUDA
n=4096, s=0,30	4,6x	14,2x ★
n=4096, s=0,50	4,5x	11,8x
n=4096, s=0,70	6,2x ★	5,5x
n=4096, s=0,85	5,0x	4,7x
n=4096, s=0,90	3,5x	3,1x
n=16384, s=0,70	3,9x	7,7x
n=32768, s=0,50	3,9x	7,4x

• Escalabilidade — $n=16384$ e $n=32768$

Hierarquia de desempenho mantida em larga escala



Observações

- MACKO FP64 atinge ~935 GB/s estável em todos os n grandes
- FP16 supera cuSPARSE em tempo absoluto em $n=16K$ e $n=32K$
- CSR-CUDA permanece em ~265 GB/s independente de n
- Gap MACKO / cuSPARSE se mantém ~2× em GB/s

• Análise: SoA não é otimização universal

CPU OpenMP vs GPU — padrões de acesso diferentes

GPU ✓ SoA vence

32 lanes do warp leem o mesmo campo de chunks vizinhos
→ 1 transação coalescida de 32B
→ valid_masks contíguos na RAM
→ Footprint -10% se soma ao ganho de coalescência

0.017 ms → 0.014 ms (-18%)

1786 GB/s → 1886 GB/s

CPU OpenMP ✗ SoA perde

Cada thread abre 4 streams de memória simultâneos (valid_masks + col_bases + col_deltas + vals)
→ Pressão × N_threads no subsistema de memória
→ AoS: tudo numa linha de cache

0.107 ms → 0.134 ms (+25%)

282 GB/s → 202 GB/s

Conclusão

01 Formato MACKO-SoA-FP16 supera o baseline

Speedup de até 14,2× vs CSR-CUDA. Bate o cuSPARSE em tempo absoluto em alta esparsidade. 1895 GB/s de pico na RTX 4090.

02 Medir antes de assumir

A v2 era uma regressão real de -18%. O padding silencioso de struct em C anulou o delta encoding — sem medir sizeof(), o erro não seria detectado.

03 SoA depende do padrão de acesso

Ganhou na GPU (mesmo campo, lanes vizinhas). Perdeu no OpenMP CPU (todos os campos, mesma thread). Não é otimização universal.

04 Trabalhos futuros

Matrizes reais de LLaMA-2 / Mistral podados · metadados INT8 · tensor cores com FP16 · multi-GPU com NVLink.

Obrigado! Perguntas?